InsightX

Database Schema v3.0 · Full Analytics Engine · Auto-Pipeline

12	22	14	2
Tables Total	Analytics Modules	New Columns Added	New Tables Added

March 2026 · Version 3.0 · Confidential

Executive Summary

This document supersedes the v2.0 schema review. It adds three major layers to InsightX: (1) a complete data science analytics engine covering every possible analytical technique for sales data, (2) two new database tables and 14 new columns required to support those analyses, and (3) a fully automatic CSV ingestion pipeline that requires zero manual input from the user after the file is uploaded.

What Changed	Details
New tables added	csv_column_mappings, analysis_results_cache
orders table additions	payment_method, acquisition_channel, return_flag, delivery_days, cost_amount, gross_margin
customers table additions	age_group, gender, clv_predicted, cohort_month, rfm_score, rfm_segment
products table additions	cost_price, supplier, gross_margin_pct, return_rate
Analytics modules	22 modules: RFM, Cohort, Basket, Forecasting, Anomaly, Segmentation, Funnel, Geo, Sentiment, Statistical + more
Auto-pipeline	CSV upload → header detection → fuzzy column mapping → validation → DB insert → all analytics triggered → dashboard refresh. Zero user input required.

Database Impact Analysis

Before any analytics can run, the schema must support the required data points. The following changes are additive — no existing columns are removed or renamed, so all current code remains unbroken.

Migration 1 — orders table: 6 new columns

> These columns unlock payment analysis, marketing attribution, return rate, margin analysis, and fulfillment performance — none of which are possible from the current schema.

Column	Type	Nullable	Purpose & Analytics Enabled
`payment_method`	`VARCHAR(50)`	YES	Credit Card, PayPal, Bank Transfer, etc. Enables payment channel analytics and conversion funnel by payment type.
`acquisition_channel`	`VARCHAR(50)`	YES	organic_search, paid_ads, referral, email, social. Enables marketing attribution and ROI analysis per channel.
`return_flag`	`BOOLEAN`	NO	TRUE = this order was returned. Enables return rate analysis by product, region, customer segment. Default FALSE.

delivery_days	INTEGER	YES	Days from order_date to delivery. Enables fulfillment SLA analytics, correlation between delivery speed and repeat purchase.
cost_amount	NUMERIC(12,2)	YES	Total cost of goods for this order. Required for gross margin = net_amount - cost_amount.
gross_margin	NUMERIC(12,2) GENERATED	NO	GENERATED ALWAYS AS (net_amount - COALESCE(cost_amount,0)) STORED. Auto-computed gross profit per order.

SQL Migration:

```
ALTER TABLE orders
   ADD COLUMN payment_method      VARCHAR(50),
   ADD COLUMN acquisition_channel  VARCHAR(50),
   ADD COLUMN return_flag          BOOLEAN NOT NULL DEFAULT FALSE,
   ADD COLUMN delivery_days        INTEGER,
   ADD COLUMN cost_amount          NUMERIC(12,2),
   ADD COLUMN gross_margin         NUMERIC(12,2)
     GENERATED ALWAYS AS (net_amount - COALESCE(cost_amount, 0)) STORED;
```

Migration 2 — customers table: 6 new columns

> These columns are required for RFM scoring, cohort retention analysis, demographic segmentation, and predictive CLV modelling.

Column	Type	Nullable	Purpose & Analytics Enabled
age_group	VARCHAR(20)	YES	18-24, 25-34, 35-44, 45-54, 55+. Enables demographic cohort analysis and age-based segmentation.
gender	VARCHAR(20)	YES	Male, Female, Non-binary, Prefer not to say. Enables gender-based purchase pattern analysis.
clv_predicted	NUMERIC(12,2)	YES	ML-predicted Customer Lifetime Value. Populated by analytics worker after each CSV upload. Separate from historical lifetime_value.
cohort_month	DATE	YES	First day of the month the customer made their first purchase. Set automatically from first_purchase_date. Core of cohort retention heatmap.
rfm_score	VARCHAR(10)	YES	Composite RFM score e.g. "555", "312". Computed by analytics worker. R=Recency, F=Frequency, M=Monetary each 1-5.
rfm_segment	VARCHAR(50)	YES	Champions, Loyal, At Risk, Cannot Lose, Hibernating, etc. Derived from rfm_score. Replaces basic ai_segment for richer segmentation.

SQL Migration:

```
ALTER TABLE customers
   ADD COLUMN age_group       VARCHAR(20),
   ADD COLUMN gender          VARCHAR(20),
   ADD COLUMN clv_predicted   NUMERIC(12,2),
   ADD COLUMN cohort_month    DATE,
   ADD COLUMN rfm_score       VARCHAR(10),
   ADD COLUMN rfm_segment     VARCHAR(50);
  -- Auto-populate cohort_month from existing data
 UPDATE customers SET cohort_month = DATE_TRUNC('month', first_purchase_date);
```

Migration 3 — products table: 4 new columns

> □ Without cost_price, margin analysis is impossible. Without supplier, supply chain analytics cannot run. Without return_rate, product quality scoring is missing.

Column	Type	Nullable	Purpose & Analytics Enabled
`cost_price`	`NUMERIC(10,2)`	YES	Unit cost of goods. Enables gross margin per product = unit_price - cost_price. Core for profitability analysis.
`supplier`	`VARCHAR(100)`	YES	Supplier / vendor name. Enables supplier performance analytics, concentration risk analysis.
`gross_margin_pct`	`NUMERIC(6,2) GENERATED`	NO	GENERATED AS ((unit_price - COALESCE(cost_price,0)) / NULLIF(unit_price,0) * 100). Auto-computed margin %.
`return_rate`	`NUMERIC(5,2)`	YES	% of orders for this product that were returned. Computed by analytics worker. Used in product quality scoring.

SQL Migration:

```
ALTER TABLE products
   ADD COLUMN cost_price        NUMERIC(10,2),
   ADD COLUMN supplier          VARCHAR(100),
   ADD COLUMN gross_margin_pct  NUMERIC(6,2)
     GENERATED ALWAYS AS (
       (unit_price - COALESCE(cost_price,0)) / NULLIF(unit_price,0) * 100
     ) STORED,
   ADD COLUMN return_rate       NUMERIC(5,2);
```

New Tables Required

New Table A: csv_column_mappings

This table is the brain of the auto-pipeline. Every CSV header from an upload is recorded here alongside the database column it maps to, the confidence of the fuzzy match, and whether it was auto-confirmed or needed a warning. The system uses this history to get smarter over time — if "Sale Amount" mapped to orders.total_amount last time, it will immediately confirm that mapping on the next upload.

Column	Type	Req?	Description
id	UUID PK	YES	Primary key
upload_job_id	UUID FK	YES	References upload_jobs(id). Links mapping to the specific upload batch.
csv_header	VARCHAR(255)	YES	The raw header string from the CSV file, e.g. "Order ID", "Cust Email", "Sale Amount".
mapped_table	VARCHAR(100)	YES	Target database table name, e.g. "orders", "customers".
mapped_column	VARCHAR(100)	YES	Target column name, e.g. "total_amount", "email".
match_score	NUMERIC(5,4)	YES	Fuzzy match confidence 0.0000–1.0000. Score >= 0.85 = auto-confirmed. Below = logged as low-confidence.
match_method	VARCHAR(30)	YES	exact | fuzzy | semantic | historical. How the mapping was determined.
is_confirmed	BOOLEAN	YES	TRUE = mapping used for insert. FALSE = column skipped (no match found above threshold).
created_at	TIMESTAMPTZ	YES	Audit timestamp.

CREATE TABLE SQL:

```
CREATE TABLE csv_column_mappings (
   id             UUID PRIMARY KEY DEFAULT gen_random_uuid(),
   upload_job_id  UUID NOT NULL REFERENCES upload_jobs(id) ON DELETE CASCADE,
   csv_header     VARCHAR(255) NOT NULL,
   mapped_table   VARCHAR(100) NOT NULL,
   mapped_column  VARCHAR(100) NOT NULL,
   match_score    NUMERIC(5,4) NOT NULL,
   match_method   VARCHAR(30)  NOT NULL DEFAULT 'fuzzy',
   is_confirmed   BOOLEAN      NOT NULL DEFAULT TRUE,
   created_at     TIMESTAMPTZ  NOT NULL DEFAULT NOW()
 );
  CREATE INDEX idx_csv_mappings_job    ON csv_column_mappings(upload_job_id);
 CREATE INDEX idx_csv_mappings_header ON csv_column_mappings(csv_header);
```

New Table B: analysis_results_cache

Every analytics module (RFM, Cohort, Basket, etc.) writes its output here as a JSON blob keyed by analysis type and the triggering upload job. The frontend reads from this cache instantly instead of recomputing. A Celery beat task invalidates stale results after 24 hours or whenever a new upload job completes.

Column	Type	Req?	Description
id	UUID PK	YES	Primary key
analysis_type	VARCHAR(60)	YES	rfm | cohort | basket | forecast | anomaly | segmentation | funnel | geo | sentiment | statistical | …
upload_job_id	UUID FK	YES	The upload job that triggered this analysis run. Allows correlating results to specific data imports.
result_json	JSONB	YES	Full analysis output as JSONB. Queried directly by the frontend API. Structure varies per analysis type.
computed_at	TIMESTAMPTZ	YES	When this result was computed. Used for cache staleness checks.
is_stale	BOOLEAN	YES	Set TRUE by Celery beat task when a new upload job invalidates this result. Frontend shows spinner while recomputing.
duration_ms	INTEGER	opt	How many milliseconds the analysis took to run. Used for performance monitoring and SLA tracking.

CREATE TABLE SQL:

```
CREATE TABLE analysis_results_cache (
   id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
   analysis_type   VARCHAR(60)  NOT NULL,
   upload_job_id   UUID REFERENCES upload_jobs(id),
   result_json     JSONB        NOT NULL,
   computed_at     TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
   is_stale        BOOLEAN      NOT NULL DEFAULT FALSE,
   duration_ms     INTEGER,
   UNIQUE(analysis_type, upload_job_id)
 );
  CREATE INDEX idx_arc_type     ON analysis_results_cache(analysis_type);
 CREATE INDEX idx_arc_stale    ON analysis_results_cache(is_stale) WHERE is_stale =
FALSE;
 CREATE INDEX idx_arc_computed ON analysis_results_cache(computed_at DESC);
```

Fully Automatic CSV Ingestion Pipeline

> □ Zero user input required after the CSV file is dropped. The system reads, maps, validates, inserts, and triggers all analytics automatically.

Pipeline Architecture — 7 Stages

Stage	Worker / Component	What Happens
1	**FastAPI Upload Endpoint**	User drops CSV. File is streamed to S3. upload_jobs row is created with status=pending. Celery task is dispatched immediately. API returns {job_id} to frontend — frontend begins polling.
2	**task: detect_columns**	CSV is read from S3. pandas reads the first row to extract headers. Each header is run through the fuzzy matching engine (RapidFuzz + semantic similarity). Every mapping is written to csv_column_mappings. upload_jobs.status → "mapping".
3	**task: validate_rows**	Every row is validated against target column types. Dates parsed with dateutil. Numerics coerced. Nulls filled with defaults. Bad rows are counted in rows_failed. upload_jobs.rows_total is set. Status → "validating".
4	**task: upsert_entities**	Customers and products are upserted first (ON CONFLICT DO UPDATE). Ensures referential integrity before orders are inserted. cohort_month auto-set from first_purchase_date. Status → "inserting".
5	**task: insert_orders**	Orders and order_items are bulk-inserted using COPY protocol (fastest PostgreSQL insert). Each row gets upload_job_id stamped. Denormalized counters on customers and products are recalculated. Status → "inserting".
6	**task: run_analytics_chain**	All 22 analytics modules fire in parallel Celery chord. Each module reads from DB, computes its result, writes to analysis_results_cache. is_stale flipped to FALSE for each. Status → "analyzing".
7	**Completion + WebSocket Push**	upload_jobs.status → "completed". rows_processed and rows_failed finalized. WebSocket event pushed to frontend. Dashboard refreshes automatically. User sees updated charts with zero manual action.

Column Fuzzy Matching Engine — How It Works

The mapping engine runs 3 strategies in sequence, escalating only if the previous step scores below the confidence threshold:

Step	Method	Example Match	Confidence Threshold
1	Exact match (case-insensitive)	"order_date" → orders.order_date	1.00 — immediately confirmed

2	RapidFuzz token ratio	"Sale Amt" → orders.total_amount (0.87)	≥ 0.85 = auto-confirmed. 0.60–0.84 = logged as low-confidence but used.
3	Semantic similarity (sentence-transformers)	"Client identifier" → customers.external_id	≥ 0.75 = confirmed. Below = column skipped, logged as unmapped.
4	Historical lookup	Previous mapping for same header reused instantly	Inherits prior score. Overrides all other methods if found.

Common CSV Header Examples — Auto-Mapped

CSV Header (any variation)	Maps To	Method	Score
"Order ID", "order_id", "OrderNo", "Txn ID"	`orders.external_id`	exact/fuzzy	0.95–1.00
"Customer Email", "Cust_Email", "email address"	`customers.email`	exact/fuzzy	0.88–1.00
"Sale Amount", "Revenue", "Total", "Amt"	`orders.total_amount`	fuzzy	0.85–0.92
"Order Date", "Purchase Date", "Txn Date"	`orders.order_date`	fuzzy	0.87–1.00
"Product SKU", "Item Code", "SKU"	`products.sku`	exact/fuzzy	0.90–1.00
"Qty", "Quantity", "Units Sold", "Count"	`order_items.quantity`	fuzzy	0.82–0.95
"Customer Name", "Full Name", "Client Name"	`customers.full_name`	fuzzy	0.86–0.98
"Country", "Nation", "Customer Country"	`customers.country`	fuzzy/semantic	0.85–0.96
"Payment Method", "Pay Type", "Payment Type"	`orders.payment_method`	fuzzy	0.88–1.00
"Discount", "Discount Amount", "Promo Deduction"	`orders.discount_amount`	fuzzy	0.80–0.94

Analytics Engine — All 22 Modules

Every module listed below is triggered automatically when a CSV upload completes. Results are written to analysis_results_cache and/or back to the relevant table columns. The dashboard reads from cache — no module re-runs on page load.

Module Overview — Quick Reference

ID	Module Name	Priority	Key Inputs	Output Destination
A01	RFM Scoring & Segmentation	CRITICAL	orders.order_date, customers.id, orders.net_amount	customers.rfm_score
A02	Cohort Retention Analysis	CRITICAL	customers.cohort_month, orders.customer_id, orders.order_date	analysis_results_cache (type=cohort)
A03	Revenue Analytics	CRITICAL	orders.net_amount, orders.order_date, orders.region	daily_kpi_snapshots
A04	Market Basket / Association Rules	HIGH	order_items.order_id, order_items.product_id, products.name	analysis_results_cache (type=basket)
A05	Time-Series Forecasting (Prophet)	HIGH	orders.order_date, orders.net_amount, forecast_results.scenario_*	forecast_results table
A06	Anomaly Detection	HIGH	orders.order_date, orders.net_amount, orders.total_amount	analysis_results_cache (type=anomaly)
A07	Customer Lifetime Value Prediction	HIGH	orders.customer_id, orders.order_date, orders.net_amount	customers.clv_predicted
A08	Sentiment Analysis (BERT NLP)	HIGH	orders.comment_text, orders.order_date	orders.sentiment_label
A09	Customer Segmentation (K-Means)	HIGH	customers.lifetime_value, customers.total_orders, customers.engagement_score	customers.ai_segment
A10	Product ABC Analysis	MEDIUM	order_items.product_id, order_items.line_total	products.abc_tier
A11	Margin & Profitability Analysis	MEDIUM	orders.gross_margin, products.cost_price, products.gross_margin_pct	analysis_results_cache (type=margin)
A12	Churn Prediction	MEDIUM	customers.last_active_at, customers.total_orders, customers.engagement_score	customers.churn_risk_score
A13	Geographic Revenue Heatmap	MEDIUM	orders.net_amount, customers.country_code, orders.region	analysis_results_cache (type=geo)
A14	Seasonality & Trend Decomposition	MEDIUM	orders.order_date, orders.net_amount	analysis_results_cache (type=seasonality)

A15	**Funnel & Repeat Purchase Analysis**	**MEDIUM**	orders.customer_id, orders.order_date, customers.total_orders	analysis_results_cache (type=funnel)
A16	**Payment Channel Analytics**	**MEDIUM**	orders.payment_method, orders.net_amount, orders.status	analysis_results_cache (type=payment_channel)
A17	**Marketing Attribution**	**MEDIUM**	orders.acquisition_channel, orders.net_amount, customers.lifetime_value	analysis_results_cache (type=attribution)
A18	**Return Rate & Product Quality**	**LOW**	orders.return_flag, orders.order_date, order_items.product_id	products.return_rate
A19	**Fulfillment Performance**	**LOW**	orders.delivery_days, orders.region, products.supplier	analysis_results_cache (type=fulfillment)
A20	**Supplier Performance Analytics**	**LOW**	products.supplier, products.total_revenue, products.return_rate	analysis_results_cache (type=supplier)
A21	**Statistical Correlation Matrix**	**LOW**	All numeric columns across orders, customers, products	analysis_results_cache (type=correlation)
A22	**Demographic Segmentation**	**LOW**	customers.age_group, customers.gender, customers.lifetime_value	analysis_results_cache (type=demographic)

Module Detail Specifications

A01 — RFM Scoring & Segmentation

Priority	CRITICAL
Description	Scores every customer on Recency (days since last order), Frequency (order count), and Monetary (total spend). Each dimension scored 1–5 using quintiles. Composite score drives 10 named segments.
DB Inputs	→ `orders.order_date` → `customers.id` → `orders.net_amount`
DB Outputs	← `customers.rfm_score` ← `customers.rfm_segment`

A02 — Cohort Retention Analysis

Priority	CRITICAL
Description	Groups customers by acquisition month (cohort_month). Tracks what % of each cohort returned to purchase in months 1, 2, 3, … 12. Output is a retention matrix that renders as a colour-coded heatmap.
DB Inputs	→ `customers.cohort_month` → `orders.customer_id` → `orders.order_date`
DB Outputs	← `analysis_results_cache (type=cohort)`

A03 — Revenue Analytics

Priority	CRITICAL
Description	Full revenue breakdown: total, MoM growth %, YoY growth %, revenue by region, by product category, by payment method, by acquisition channel, and revenue per active customer. All deltas stored as % change.
DB Inputs	→ `orders.net_amount` → `orders.order_date` → `orders.region` → `orders.acquisition_channel`
DB Outputs	← `daily_kpi_snapshots` ← `analysis_results_cache (type=revenue)`

A04 — Market Basket / Association Rules

Priority	HIGH

Description	Apriori algorithm on order_items. Finds product pairs and triplets that are frequently bought together. Outputs support, confidence, and lift scores. Powers "Customers also bought" and bundle recommendations.
DB Inputs	→ `order_items.order_id` → `order_items.product_id` → `products.name`
DB Outputs	← `analysis_results_cache (type=basket)`

A05 — Time-Series Forecasting (Prophet)

Priority	**HIGH**
Description	Facebook Prophet model fits historical daily revenue. Produces 90-day forward prediction with upper/lower confidence bands. Scenario parameters (marketing spend, price shift, seasonal adjustment) applied as multipliers.
DB Inputs	→ `orders.order_date` → `orders.net_amount` → `forecast_results.scenario_*`
DB Outputs	← `forecast_results table`

A06 — Anomaly Detection

Priority	**HIGH**
Description	Isolation Forest + Z-score hybrid on daily revenue, order volume, and average order value. Flags statistical outliers (spikes and drops). Each anomaly stored with severity score, date, and affected metric.
DB Inputs	→ `orders.order_date` → `orders.net_amount` → `orders.total_amount`
DB Outputs	← `analysis_results_cache (type=anomaly)`

A07 — Customer Lifetime Value Prediction

Priority	**HIGH**
Description	BG/NBD model (Buy-Till-They-Die) or Pareto/NBD to predict future purchase probability and expected revenue per customer over the next 6 and 12 months. Written to customers.clv_predicted.
DB Inputs	→ `orders.customer_id` → `orders.order_date` → `orders.net_amount`
DB Outputs	← `customers.clv_predicted` ← `analysis_results_cache (type=clv)`

A08 — Sentiment Analysis (BERT NLP)

Priority	HIGH
Description	Pre-trained BERT model processes orders.comment_text. Outputs POSITIVE / NEGATIVE / NEUTRAL label plus confidence score. Sentiment trend over time and breakdown by product category stored in cache.
DB Inputs	→ orders.comment_text → orders.order_date
DB Outputs	← orders.sentiment_label ← orders.sentiment_score ← analysis_results_cache (type=sentiment)

A09 — Customer Segmentation (K-Means)

Priority	HIGH
Description	K-Means clustering (k=5 by default, auto-tuned via elbow method) on scaled features: lifetime_value, total_orders, engagement_score, recency_days, clv_predicted. Cluster centroids stored for bubble chart.
DB Inputs	→ customers.lifetime_value → customers.total_orders → customers.engagement_score
DB Outputs	← customers.ai_segment ← analysis_results_cache (type=segmentation)

A10 — Product ABC Analysis

Priority	MEDIUM
Description	Ranks products by cumulative revenue contribution. Top 20% of revenue → A tier. Next 30% → B tier. Remaining 50% → C tier. Updates products.abc_tier and products.abc_computed_at.
DB Inputs	→ order_items.product_id → order_items.line_total
DB Outputs	← products.abc_tier ← products.abc_computed_at

A11 — Margin & Profitability Analysis

Priority	MEDIUM
Description	Computes gross margin per order, per product, per category, and per customer segment. Identifies highest and lowest margin products. Margin trend over time. Requires cost_price data in CSV.

DB Inputs	→ `orders.gross_margin` → `products.cost_price` → `products.gross_margin_pct`
DB Outputs	← `analysis_results_cache (type=margin)`

A12 — Churn Prediction

Priority	**MEDIUM**
Description	Logistic regression model trained on: days_since_last_order, total_orders, avg_order_gap, engagement_score, rfm_score. Outputs churn probability 0–1 written to customers.churn_risk_score.
DB Inputs	→ `customers.last_active_at` → `customers.total_orders` → `customers.engagement_score` → `customers.rfm_score`
DB Outputs	← `customers.churn_risk_score` ← `analysis_results_cache (type=churn)`

A13 — Geographic Revenue Heatmap

Priority	**MEDIUM**
Description	Aggregates net_amount by customers.country_code. Returns a GeoJSON-ready structure with revenue per country for the choropleth map. Also breaks down by region (NA/EU/APAC/LATAM) for bar chart.
DB Inputs	→ `orders.net_amount` → `customers.country_code` → `orders.region`
DB Outputs	← `analysis_results_cache (type=geo)`

A14 — Seasonality & Trend Decomposition

Priority	**MEDIUM**
Description	STL decomposition (Seasonal-Trend decomposition using LOESS) on daily revenue. Separates trend, seasonality, and residual. Identifies day-of-week patterns, monthly peaks, and holiday effects.
DB Inputs	→ `orders.order_date` → `orders.net_amount`
DB Outputs	← `analysis_results_cache (type=seasonality)`

A15 — Funnel & Repeat Purchase Analysis

Priority	MEDIUM
Description	Tracks the conversion funnel: New Customer → 2nd Purchase → 3rd Purchase → Loyal (5+). Computes median time between purchases, repeat purchase rate %, and funnel drop-off at each stage.
DB Inputs	→ `orders.customer_id` → `orders.order_date` → `customers.total_orders`
DB Outputs	← `analysis_results_cache (type=funnel)`

A16 — Payment Channel Analytics

Priority	MEDIUM
Description	Breakdown of orders and revenue by orders.payment_method. Conversion rate per payment type, average order value by payment method, and failed payment rate (if status=cancelled correlated with payment_method).
DB Inputs	→ `orders.payment_method` → `orders.net_amount` → `orders.status`
DB Outputs	← `analysis_results_cache (type=payment_channel)`

A17 — Marketing Attribution

Priority	MEDIUM
Description	Revenue and customer acquisition breakdown by orders.acquisition_channel. ROI per channel (if cost data provided). First-touch attribution model. Channel contribution to LTV.
DB Inputs	→ `orders.acquisition_channel` → `orders.net_amount` → `customers.lifetime_value`
DB Outputs	← `analysis_results_cache (type=attribution)`

A18 — Return Rate & Product Quality

Priority	LOW
Description	Aggregates orders.return_flag by product, category, region, and time period. Writes return_rate back to products table. Identifies high-return segments and correlates returns with sentiment scores.
DB Inputs	→ `orders.return_flag` → `orders.order_date` → `order_items.product_id`
DB Outputs	← `products.return_rate`

	← analysis_results_cache (type=returns)

A19 — Fulfillment Performance

Priority	LOW
Description	Analyses orders.delivery_days distribution. P50, P90, P99 delivery times by region, product category, and supplier. Correlates delivery speed with repeat purchase rate and customer satisfaction score.
DB Inputs	→ orders.delivery_days → orders.region → products.supplier
DB Outputs	← analysis_results_cache (type=fulfillment)

A20 — Supplier Performance Analytics

Priority	LOW
Description	Groups products by supplier. Computes total revenue, return rate, margin, and stock turnover per supplier. Identifies supplier concentration risk (% of revenue from single supplier).
DB Inputs	→ products.supplier → products.total_revenue → products.return_rate
DB Outputs	← analysis_results_cache (type=supplier)

A21 — Statistical Correlation Matrix

Priority	LOW
Description	Pearson correlation matrix across all numeric KPIs: revenue, engagement_score, churn_risk_score, clv_predicted, total_orders, delivery_days, return_rate. Identifies which metrics are strongly correlated.
DB Inputs	→ All numeric columns across orders, customers, products
DB Outputs	← analysis_results_cache (type=correlation)

A22 — Demographic Segmentation

Priority	LOW
Description	Cross-tabulation of revenue, order count, and LTV by customers.age_group and customers.gender. Identifies highest-value demographic segments. Powers targeted marketing insights.

DB Inputs	→ customers.age_group → customers.gender → customers.lifetime_value
DB Outputs	← analysis_results_cache (type=demographic)

Celery Task Dependency Graph

All tasks are chained using Celery chord and chain primitives. Stages 1–5 are sequential (order matters for FK integrity). Stage 6 is a parallel chord — all 22 analytics modules fire simultaneously.

Task Name	Queue	Depends On
`upload.detect_columns`	csv_queue	S3 file exists
`upload.validate_rows`	csv_queue	detect_columns SUCCESS
`upload.upsert_entities`	db_queue	validate_rows SUCCESS
`upload.insert_orders`	db_queue	upsert_entities SUCCESS
`analytics.rfm`	analytics_queue	insert_orders SUCCESS
`analytics.cohort`	analytics_queue	insert_orders SUCCESS
`analytics.revenue`	analytics_queue	insert_orders SUCCESS
`analytics.basket`	analytics_queue	insert_orders SUCCESS
`analytics.forecast`	ml_queue	insert_orders SUCCESS
`analytics.anomaly`	ml_queue	insert_orders SUCCESS
`analytics.clv`	ml_queue	insert_orders SUCCESS
`analytics.sentiment`	ml_queue	insert_orders SUCCESS
`analytics.segmentation`	ml_queue	analytics.rfm SUCCESS
`analytics.churn`	ml_queue	analytics.rfm SUCCESS
`analytics.abc_tier`	analytics_queue	insert_orders SUCCESS
`analytics.margin`	analytics_queue	insert_orders SUCCESS
`analytics.geo`	analytics_queue	insert_orders SUCCESS
`analytics.seasonality`	analytics_queue	insert_orders SUCCESS
`analytics.funnel`	analytics_queue	insert_orders SUCCESS
`analytics.payment_channel`	analytics_queue	insert_orders SUCCESS
`analytics.attribution`	analytics_queue	insert_orders SUCCESS
`analytics.returns`	analytics_queue	insert_orders SUCCESS
`analytics.fulfillment`	analytics_queue	insert_orders SUCCESS
`analytics.supplier`	analytics_queue	analytics.returns SUCCESS
`analytics.correlation`	analytics_queue	All analytics SUCCESS
`analytics.demographic`	analytics_queue	insert_orders SUCCESS
`upload.finalize`	db_queue	All analytics SUCCESS

Updated Schema Summary — 11 Tables

The table below reflects the complete final state of the InsightX database after all migrations in this document are applied.

Table	Status	Key New Columns	Primary Role
`users`	UNCHANGED	—	Auth, roles, widget layout per user
`customers`	MODIFIED	**+6 columns**	Customer profiles, RFM scores, CLV, cohort, segmentation
`products`	MODIFIED	**+4 columns**	Product catalogue, ABC tier, margin, return rate, supplier
`orders`	MODIFIED	**+6 columns**	Central fact table, revenue, margins, payment, channel, returns
`order_items`	UNCHANGED	—	Line items per order, stock deduction, basket analysis
`upload_jobs`	UNCHANGED	—	CSV upload tracking, progress, lineage
`csv_column_mappings`	NEW	**All columns new**	Fuzzy mapping engine output; auto-pipeline header mapping log
`forecast_results`	UNCHANGED	—	Prophet time-series output with scenario params
`daily_kpi_snapshots`	UNCHANGED	—	Pre-aggregated KPI snapshots for period comparisons
`analysis_results_cache`	NEW	**All columns new**	Cache for all 22 analytics modules; invalidated on new upload
`insights`	UNCHANGED	—	LLaMA 3 generated bullet insights per analysis run

Updated ER Diagram — Relationships

```
users            ||--o{ upload_jobs              : "triggers"
 users            ||--o{ forecast_results         : "runs"
 customers        ||--o{ orders                   : "places"
 upload_jobs      ||--o{ orders                   : "imports"
 upload_jobs      ||--o{ csv_column_mappings      : "produces"
 upload_jobs      ||--o{ analysis_results_cache   : "triggers"
 orders           ||--o{ order_items              : "contains"
 products         ||--o{ order_items              : "appears_in"
```

Implementation Checklist

✓	Action Required	Owner	Notes
☐	Run Migration 1 — ALTER TABLE orders (6 columns)	DBA	Run in a transaction; verify GENERATED column computes correctly on test row
☐	Run Migration 2 — ALTER TABLE customers (6 columns)	DBA	Run UPDATE to backfill cohort_month from existing first_purchase_date values
☐	Run Migration 3 — ALTER TABLE products (4 columns)	DBA	GENERATED gross_margin_pct will be NULL for rows where cost_price is NULL
☐	CREATE TABLE csv_column_mappings	DBA	ON DELETE CASCADE ensures mappings are cleaned up if upload_job is rolled back
☐	CREATE TABLE analysis_results_cache	DBA	UNIQUE(analysis_type, upload_job_id) enforced — no duplicate results per run
☐	Deploy fuzzy mapping Celery worker	Backend Dev	Install: rapidfuzz, sentence-transformers. Worker: csv_queue
☐	Deploy analytics Celery workers (×22)	Data Science	Install: scikit-learn, lifetimes, mlxtend, prophet, statsmodels, transformers
☐	Wire upload_jobs WebSocket push on completion	Backend Dev	Frontend listens for {job_id, status: "completed"} → triggers dashboard refresh
☐	Add new columns to CSV mapping config	Backend Dev	payment_method, acquisition_channel, return_flag, delivery_days, cost_amount
☐	Backfill existing orders with analytics	Data Science	Run run_analytics_chain manually once after migrations to populate cache
☐	Update API endpoints to serve cache	Backend Dev	All analytics endpoints read analysis_results_cache first; compute only if is_stale
☐	Update dashboard UI for new analytics tabs	Frontend Dev	Add: RFM heatmap, Cohort grid, Basket chart, Margin waterfall, Anomaly timeline